

Vedak 实现代码说明

本目录包含密码杂凑算法 Vedak 的三类软件实现、配套测试向量以及接口代码。所有实现均遵循 CryptHash 编程接口，支持 Vedak-512、Vedak-768 和 Vedak-1024 三个实例。三类实现的算法行为完全等价，在相同输入下输出完全一致；它们的差别仅在于实现方式、适用平台与运行性能。

1. 总体说明

本提交包提供以下三类实现：

- `Reference_Implementation`：参考实现，采用标准 C 语言编写，不依赖 AVX2、AVX-512 等专用向量指令，主要用于规范验证、正确性测试与可移植性编译。
- `Optimized_Implementation`：优化实现，采用 AVX2（256 比特向量）指令进行优化，适用于绝大多数支持 AVX2 的主流 PC 和服务端平台。
- `Additional_Implementation`：附加实现，采用 AVX-512 指令集进行进一步优化，仅适用于支持 AVX-512 的高性能平台。

建议的使用顺序如下：若平台支持 AVX-512，则可优先使用 `Additional_Implementation`；若不支持，则使用 `Optimized_Implementation`；若需要最高可移植性或进行独立验证，则使用 `Reference_Implementation`。

2. 目录结构

API_CryptHash/

└─ Implementations/

│ └─ Reference_Implementation/

│ │ └─ Vedak-512/

│ │ └─ Vedak-768/

│ │ └─ Vedak-1024/

│ └─ Optimized_Implementation/

│ │ └─ Vedak-512/

│ │ └─ Vedak-768/

│ │ └─ Vedak-1024/

│ └─ Additional_Implementation/

│ └─ Vedak-512/

```
|   |— Vedak-768/
|   |— Vedak-1024/
|   |— Test_Vector/
|       |— KAT_2_12_Vedak-512.txt
|       |— KAT_2_23_Vedak-512.txt
|       |— KAT_2_33_Vedak-512.txt
|       |— KAT_Loop_Vedak-512.txt
|       |— KAT_2_12_Vedak-768.txt
|       |— KAT_2_23_Vedak-768.txt
|       |— KAT_2_33_Vedak-768.txt
|       |— KAT_Loop_Vedak-768.txt
|       |— KAT_2_12_Vedak-1024.txt
|       |— KAT_2_23_Vedak-1024.txt
|       |— KAT_2_33_Vedak-1024.txt
|       |— KAT_Loop_Vedak-1024.txt
```

每个实例目录均为自包含目录，可单独编译、单独运行，并生成当前实例对应的测试向量文件。

3. 算法实例

实例	输出长度	比特率 r	容量 c
Vedak-512	512 比特	1536 比特	1024 比特
Vedak-768	768 比特	1024 比特	1536 比特
Vedak-1024	1024 比特	512 比特	2048 比特

三个实例分别在三类实现目录中给出对应代码，接口形式保持一致。

4. 文件说明

- **CryptHash_AlgorithmInstance.c**：当前实例的具体实现文件，定义 CryptHash 接口函数以及内部置换、填充、吸收和挤出过程。
- **CryptHash_AlgorithmInstance.h**：接口头文件，定义算法实例名称、摘要长度、宏配置及 CryptHash 函数原型。
- **KAT_CryptHash.c**：已知答案测试（KAT）生成程序，用于生成 KAT_2_12、KAT_2_23、KAT_2_33 和 KAT_Loop 文件。

- **drng.c / drng.h**：KAT 生成程序使用的确定性随机数发生器实现及其头文件。

5. 编程接口

所有实现均提供统一接口：

```
int CryptHash(int digest_len_bits,  
              const unsigned char *msg,  
              unsigned long long msg_len_bits,  
              unsigned char *digest);
```

- **digest_len_bits**：请求输出的摘要长度，必须与当前实例目录匹配。例如 Vedak-512 目录中仅接受 512。
- **msg**：输入消息指针。空消息情况下，仅当 msg_len_bits 为 0 时，该指针可以为 NULL。
- **msg_len_bits**：输入消息长度，单位为比特。实现支持按比特计长的消息。
- **digest**：输出缓冲区指针，其大小至少应为 digest_len_bits / 8 字节。
- **返回值**：执行成功返回 0，发生参数错误或实例不匹配时返回非 0 值。

6. 编译环境与运行平台

建议使用 64 位编译环境，以便顺利生成大规模测试向量文件并获得更稳定的运行表现。推荐编译器包括 GCC、Clang、MSYS2/MinGW-w64 下的 GCC，以及 Microsoft Visual Studio。

6.1 Reference_Implementation

该版本不依赖专用向量指令，适合功能验证、调试和可移植性测试。

```
gcc -O2 -std=c99 -Wall -Wextra KAT_CryptHash.c drng.c  
CryptHash_AlgorithmInstance.c -o kat
```

```
gcc -O2 -std=c99 -Wall -Wextra KAT_CryptHash.c drng.c  
CryptHash_AlgorithmInstance.c -o kat.exe
```

6.2 Optimized_Implementation

该版本采用 AVX2 指令优化，适用于大多数支持 AVX2 的主流 x86-64 平台。

```
gcc -O3 -std=c99 -Wall -Wextra -mavx2 -march=native KAT_CryptHash.c drng.c  
CryptHash_AlgorithmInstance.c -o kat
```

```
gcc -O3 -std=c99 -Wall -Wextra -mavx2 -march=native KAT_CryptHash.c drng.c
```

```
CryptHash_AlgorithmInstance.c -o kat.exe
```

6.3 Additional Implementation

该版本采用 AVX-512 指令集，仅可在支持 AVX-512 的目标平台上编译和运行。若 CPU 或操作系统不支持相关指令集，则不应使用该版本。

```
gcc -O3 -std=c99 -Wall -Wextra -mavx512f -mavx512bw -mavx512vl -march=native
```

```
KAT_CryptHash.c drng.c CryptHash_AlgorithmInstance.c -o kat
```

```
gcc -O3 -std=c99 -Wall -Wextra -mavx512f -mavx512bw -mavx512vl -march=native
```

```
KAT_CryptHash.c drng.c CryptHash_AlgorithmInstance.c -o kat.exe
```

如果平台不支持 AVX-512，应退回到 Optimized_Implementation；如果平台不支持 AVX2，则使用 Reference_Implementation。

7. 关于 Warning 的说明

在 Microsoft Visual Studio 环境下，可能出现诸如 fopen、strcpy 等函数的 C4996 警告。这类警告通常来自 Microsoft CRT 的安全提示机制，不影响算法逻辑正确性。若希望在代码层面直接消除此类警告，可在相关源文件最前面加入下述宏定义：

```
#define _CRT_SECURE_NO_WARNINGS
```

若项目采用 Visual Studio 工程方式管理，也可在项目的预处理器定义中加入 _CRT_SECURE_NO_WARNINGS。